

Week 2 - Day 7a Supplementary: More Recursion

Question 1: Greatest Common Divisor []

Write a recursive function `gcd(a, b)` that:

1. Takes two positive integers `a` and `b` as input.
2. Returns the **greatest common divisor** of `a` and `b`.

For example:

```
gcd(15, 10) == 5 # nothing bigger than 5 divides 15 and 10.
gcd(24, 36) == 12
gcd(72, 180) == 36
gcd(101, 197) == 1 # 101 and 197 do not share any common factors.
gcd(39793, 1) == 1 # gcd(a, 1) == 1 for all positive a.
gcd(22, 0) == 22 # gcd(a, 0) == a for all positive a.
```

To implement `gcd()`, we will use a recursive algorithm that Euclid discovered in Alexandria, Egypt in 300BC. Here it is:

Recursive case:

If $b > 0$, `gcd(a, b) == gcd(b, a % b)`

Base case

`gcd(a, 0) == a`

Here is an example of Euclid's algorithm for `gcd(180, 72)`:

```
# Recursive case: 180 % 72 == 36
gcd(180, 72) == gcd(72, 36)

# Recursive case: 72 % 36 == 0
gcd(72, 36) == gcd(36, 0)

# Base case (b == 0)
gcd(36, 0) == 36
```

Result: the greatest common divisor of 180 and 72 is 36.

Here is another example, of `gcd(45, 210)`:

```

# Recursive case: 45 % 210 == 45
gcd(45, 210) == gcd(210, 45)

# Recursive case: 210 % 45 == 30
gcd(210, 45) == gcd(45, 20)

# Recursive case: 45 % 30 == 15
gcd(45, 30) == gcd(30, 15)

# Recursive case: 30 % 15 == 0
gcd(30, 15) == gcd(15, 0)

# Base case (b == 0)
gcd(15, 0) == 15

```

Result: the greatest common divisor of 45 and 210 is 15.

Task: Implement `gcd()` using Euclid's algorithm described above.

```

# TODO: Implement greatest common divisor using Euclid's algorithm
def gcd(a, b):
    # YOUR ANSWER HERE
    pass

# After you finish, these should all print True
print(gcd(15, 10) == 5)
print(gcd(24, 36) == 12)
print(gcd(72, 180) == 36)
print(gcd(101, 197) == 1)
print(gcd(39793, 1) == 1)
print(gcd(22, 0) == 22)

```

Question 2: All Permutations! [] [] []

A **permutation** of a list is the same list, but in a different order.

For example, `[0, 2, 1]` and `[2, 0, 1]` are both permutations of `[0, 1, 2]`.

Write a recursive function `permutations(n)` that takes an integer, `n`, as input, and returns a **sorted** list of all permutations of integers between 0 and `n`. Examples:

```

permutations(0) == [[0]]

permutations(1) == [[0, 1], [1, 0]]

permutations(2) == [[0, 1, 2], [0, 2, 1],
                    [1, 0, 2], [1, 2, 0],
                    [2, 0, 1], [2, 1, 0]]

```

Hints

- `myList.insert(5, 'myItem')` will insert 'myItem' at position 5 in `myList`.
- Use `myList.sort()` to put `myList` in sorted order. Note: `sort()` returns `None`, so use it on its own line of code.
- Here, it is OK to use loops too as part of your **recursive solution**.

```
# Example code to illustrate `insert()` and `sort()`

myList = [2, 0, 3, 1]

myList.insert(2, 77) # insert 77 at position 2 in myList
print("myList hasn't been sorted yet:", myList)

myList.sort()
print("Now myList is sorted:", myList)
```

Task: Write a recursive `permutations(n)` function.

Remember: it is OK to use loops as part of your recursive solution!

```
# TODO: Implement a function that returns all sorted permutations of
the numbers 0 through n
def permutations(n):
    # YOUR ANSWER HERE
    pass

# If your code is correct, these should all print True
print(permutations(0) == [[0]])
print(permutations(1) == [[0, 1], [1, 0]])

print(permutations(2) == [[0, 1, 2], [0, 2, 1],
                          [1, 0, 2], [1, 2, 0],
                          [2, 0, 1], [2, 1, 0]])

if permutations(0) != None:
    print(len(permutations(6)) == 5040)
    print(permutations(6)[3791] == [5, 1, 3, 6, 4, 2, 0])
else:
    print(False)
```

Question 3: Combinations Craze [] [] [] []

Consider a recursive function called `combinations`.

- It takes a list of numbers and a number `k`.
- It returns all the possible combinations of size `k`.

Combinations of size `k` refer to the subsets or groupings of elements taken from a given set, where each combination consists of exactly `k` elements. In other words, it represents the selection of `k` distinct elements from a larger set, without considering the order of the elements.

For example:

```
combinations([1, 2, 3], 2) == [[1, 2], [1, 3], [2, 3]]
combinations([4, 5, 6], 3) == [[4, 5, 6]]
combinations([1, 2, 3, 4], 5) == []
```

Think about:

- What is the recursive case for `combinations()`? Describe your answer using English.
- What is the base case for `combinations()`? Describe your answer using English.

Implement `combinations()`.

```
# TODO: Implement a function that returns all combinations of k
elements from a list
def combinations(lst, k):
    # YOUR ANSWER HERE
    pass

#Test cases
print('Test passed:', combinations([1, 2, 3], 2) == [[1, 2], [1, 3],
[2, 3]])
print('Test passed:', combinations([4, 5, 6], 3) == [[4, 5, 6]])
print('Test passed:', combinations([4, 5, 6], 2) == [[4, 5], [4, 6],
[5, 6]])
print('Test passed:', combinations([1, 2, 3, 4], 5) == [])
```

Congratulations 